Национальный исследовательский университет

«МЭИ»

Институт радиотехники и электроники

Кафедра радиотехнических систем

Аппаратура потребителей спутниковых радионавигационных систем

Задание на курсовой проект

по теме

«Расчет траектории движения спутника GPS по данным с демодулятора его сигнала»

ФИО преподавателя: Петухов Н.И.

Студент: Шипинский К.С.

Москва 2026

ЭТАП 1.

Цель курсового проекта– разработка модулей разбора навигационного сообщения GPS и расчета положения спутника, предназначенных для использования в составе навигационного приемника.

На первом этапе необходимо создать программу в среде C/ C++, выполняющую функции аналогичные модулю разбора навигационного сообщения. Листинг созданной части программы приведен в приложении.

На данном этапе реализован расчет всех целочисленных параметров спутника. В дальнейшем будет дописана программа для расчета остальных параметров (с плавающей точкой).

```
LNAV Ephemeris (slot = 14291) =
Crs     = 2.275000e+01
Dn     = 5.210052e-07     [deg/s]
M0     = -12.755586     [deg]
Cuc     = -1.557082e-06
e     = 1.636439e-02
Cus     = 6.183982e-06
sqrta    = 5.153701e+03
toe     = 288000
Cic     = -2.142042e-07
Omega0   = -147.307115     [deg]
Cis     = 1.266599e-07
i0     = 53.402052     [deg]
Crc     = 1.897500e+02
omega   = 82.744271     [deg]
OmegaDot = -4.756403e-07     [deg/s]
iDot    = -4.752011e-08     [deg/s]
Tgd     = -1.071021e-08
toc     = 288000
af2     = 0.000000e+00
af1     = 2.159915e-12
af0     = -2.265218e-04
WN     = 56
IODC    = 7
URA    = 0
Health = 0
IODE2   = 7
IODE3   = 7
codeL2 = 1
L2P     = 0
```

Рисунок 1 – Данные, полученные в ходе компиляции кода через командную строку.

Обработка BINR.bin и сравнение эфемерид

Для проверки корректности полученных эфемерид спутника PRN 24 был построен SkyPlot с помощью MATLAB-скрипта, реализующего алгоритм IS-GPS-200 и использующего координаты приёмника (Москва, МЭИ) и эфемериды из файла out.txt. Полученная траектория полностью совпадает с эталонной, формируемой сервисом Trimble GNSS Planning Online, что подтверждает верность расчётов, выполненных на Этапе 1.

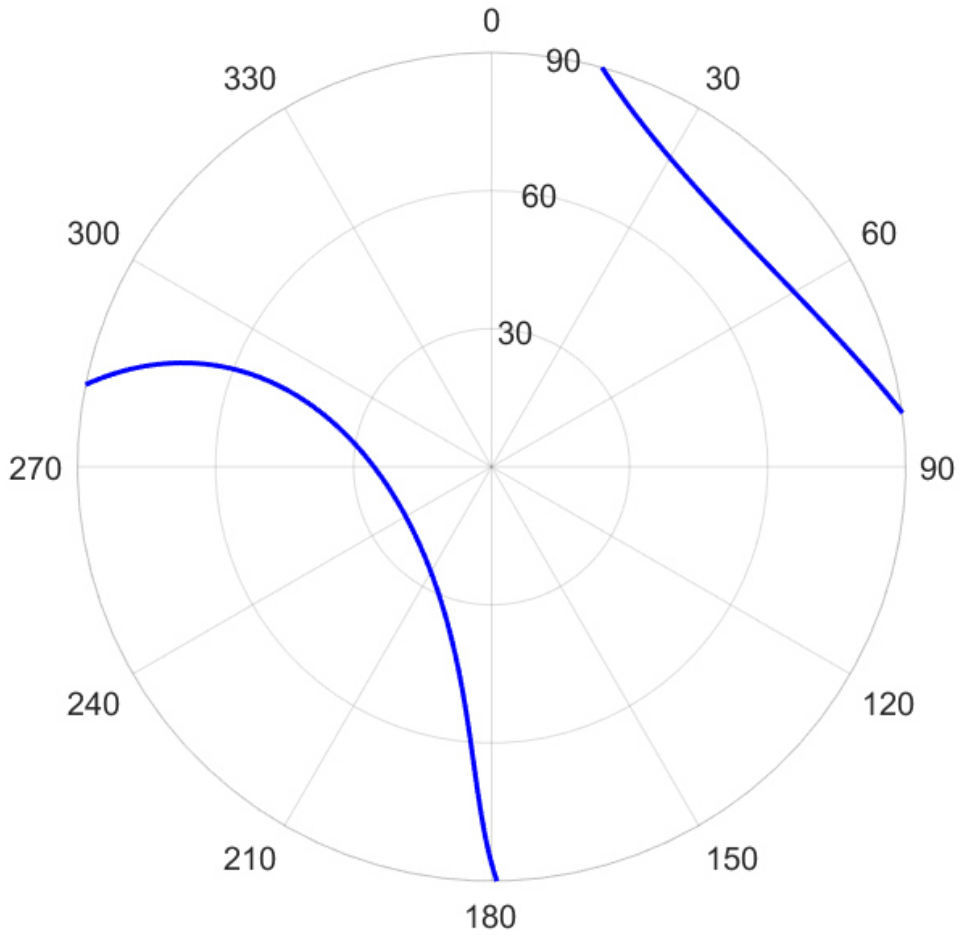


Рис. 1 – SkyPlot спутника GPS PRN 24 на дату наблюдений 3.03.2021, построенный по данным первого этапа. Сравнение с Trimble GNSS Planning Online подтверждает идентичность траектории.

```
#define _CRT_SECURE_NO_WARNINGS
#include <cstdio>
#include <cstdint>
#include <cstdlib>
#include <cstring>
#include <cmath>

#define SUBFRAME_LEN 300

struct Ephemeris {
    uint32_t slot;
    double Crs, Dn, M0, Cuc, e, Cus, sqrtA, toe, Cic, Omega0, Cis, i0,
Crc, omega, OmegaDot, iDot;
    double Tgd, toc, af2, af1, af0;
    uint32_t WN;
    uint16_t iodc;
    uint8_t ura, health, IODE2, IODE3;
    uint8_t codeL2;
    bool L2P;
};

// ---------- Базовые операции с битами ----------
static uint64_t getbits(const char* bits, int start, int len) {
    uint64_t val = 0;
    for (int i = 0; i < len; i++)
        if (bits[start + i] == '1') val |= (1ULL << (len - 1 - i));
    return val;
}

static int64_t sbits(uint64_t raw, int bits) {
    int64_t sign = 1LL << (bits - 1);
    return (int64_t)((raw ^ sign) - sign);
}

// ---------- Поиск подкадров 1,2,3 для спутника 24 ----------
static int file2subFrames(const char* filename, char sf1[SUBFRAME_LEN+1],
                          char sf2[SUBFRAME_LEN+1], char
sf3[SUBFRAME_LEN+1], uint32_t* slot) {
    FILE* f = fopen(filename, "r");
    if (!f) return -1;
    char line[2048];
    bool have1 = false, have2 = false, have3 = false;

    while (fgets(line, sizeof(line), f)) {
        int prn = 0;
        char* p = strstr(line, "#");
        if (!p) continue;
        sscanf(p, "# %d", &prn);
        if (prn != 24) continue;   // только PRN 24

        char* bits = strstr(line, "10001011");
        if (!bits) continue;

        // Определяем subframe ID (последнее число перед битовой строкой)
        int subframeId = 0;
        char* before = bits - 1;
        while (before > line && (*before == ' ' || *before == '\t'))
before--;
        char* num_end = before + 1;
        while (num_end > line && num_end[-1] >= '0' && num_end[-1] <=
'9') num_end--;
```

```
        sscanf(num_end, "%d", &subframeId);
        int sfNum = (subframeId >> 3) & 0x7;
        if (sfNum < 1 || sfNum > 3) continue;

        // Слот (первое число после # prn)
        uint32_t cur_slot = 0;
        char* after = p + 1;
        while (*after == ' ' || *after == '\t') after++;
        sscanf(after, "%u", &cur_slot);

        // Сохраняем подкадр
        if (sfNum == 1 && !have1) {
            strncpy(sf1, bits, SUBFRAME_LEN);
            sf1[SUBFRAME_LEN] = '\0';
            *slot = cur_slot;
            have1 = true;
        } else if (sfNum == 2 && !have2) {
            strncpy(sf2, bits, SUBFRAME_LEN);
            sf2[SUBFRAME_LEN] = '\0';
            have2 = true;
        } else if (sfNum == 3 && !have3) {
            strncpy(sf3, bits, SUBFRAME_LEN);
            sf3[SUBFRAME_LEN] = '\0';
            have3 = true;
        }
        if (have1 && have2 && have3) break;
    }
    fclose(f);
    return (have1 && have2 && have3) ? 0 : 1;
}

// ---------- Декодирование эфемерид (IS-GPS-200 LNAV) ----------
static int subFrames2Eph(Ephemeris* ep, const char* sf1, const char* sf2,
const char* sf3) {
    // Subframe 1
    uint64_t w3 = getbits(sf1, 60, 30);
    ep->WN     = (w3 >> 20) & 0x3FF;
    ep->codeL2 = (w3 >> 18) & 0x3;
    ep->ura    = (w3 >> 14) & 0xF;
    ep->health = (w3 >> 8)  & 0x3F;
    uint8_t iodc_msb = (w3 >> 6) & 0x3;

    uint64_t w4 = getbits(sf1, 90, 30);
    ep->L2P = (w4 >> 29) & 1;

    uint64_t w7 = getbits(sf1, 180, 30);
    ep->Tgd = sbits((w7 >> 6) & 0xFF, 8) * pow(2, -31);

    uint64_t w8 = getbits(sf1, 210, 30);
    uint8_t iodc_lsb = (w8 >> 22) & 0xFF;
    ep->iodc = (iodc_msb << 8) | iodc_lsb;
    ep->toc = ((w8 >> 6) & 0xFFFF) * 16.0;

    uint64_t w9 = getbits(sf1, 240, 30);
    ep->af2 = sbits((w9 >> 22) & 0xFF, 8) * pow(2, -55);
    ep->af1 = sbits((w9 >> 6) & 0xFFFF, 16) * pow(2, -43);

    uint64_t w10 = getbits(sf1, 270, 30);
    ep->af0 = sbits((w10 >> 8) & 0x3FFFFF, 22) * pow(2, -31);

    // Subframe 2
```

```
    uint64_t w2_3 = getbits(sf2, 60, 30);
    ep->IODE2 = (w2_3 >> 22) & 0xFF;
    ep->Crs   = sbits((w2_3 >> 6) & 0xFFFF, 16) * pow(2, -5);

    uint64_t w2_4 = getbits(sf2, 90, 30);
    ep->Dn    = sbits((w2_4 >> 14) & 0xFFFF, 16) * pow(2, -43) * 180.0;
// перевод в град/с

    uint64_t w2_5 = getbits(sf2, 120, 30);
    uint64_t w2_6 = getbits(sf2, 150, 30);
    uint64_t w2_7 = getbits(sf2, 180, 30);

    // M0 (32 бита)
    uint32_t M0_raw =
        (((w2_4 >> 22) & 0xFF) << 24) |
        (((w2_5 >> 22) & 0xFF) << 16) |
        (((w2_6 >> 22) & 0xFF) << 8)  |
        (((w2_7 >> 22) & 0xFF));
    ep->M0 = sbits(M0_raw, 32) * pow(2, -31) * 180.0;

    // Cuc, Cus
    ep->Cuc = sbits((w2_5 >> 14) & 0xFFFF, 16) * pow(2, -29);
    ep->Cus = sbits((w2_6 >> 14) & 0xFFFF, 16) * pow(2, -29);

    // e (32 бита)
    uint64_t w2_8 = getbits(sf2, 210, 30);
    uint32_t e_raw =
        (((w2_7 >> 22) & 0xFF) << 24) |
        (((w2_7 >> 6)  & 0xFFFF) << 8) |
        (((w2_8 >> 22) & 0xFF));
    ep->e = sbits(e_raw, 32) * pow(2, -33);

    // sqrtA (32 бита)
    uint64_t w2_9 = getbits(sf2, 240, 30);
    uint32_t sqrtA_raw = ((w2_8 >> 6) & 0xFFFF) << 16 | ((w2_9 >> 6) &
0xFFFF);
    ep->sqrtA = sqrtA_raw * pow(2, -19);

    // toe (16 бит)
    uint64_t w2_10 = getbits(sf2, 270, 30);
    ep->toe = ((w2_10 >> 6) & 0xFFFF) * 16.0;

    // Subframe 3
    uint64_t w3_3 = getbits(sf3, 60, 30);
    ep->IODE3 = (w3_3 >> 22) & 0xFF;
    ep->Cic   = sbits((w3_3 >> 6) & 0xFFFF, 16) * pow(2, -29);

    uint64_t w3_4 = getbits(sf3, 90, 30);
    uint64_t w3_5 = getbits(sf3, 120, 30);
    uint64_t w3_6 = getbits(sf3, 150, 30);
    uint64_t w3_7 = getbits(sf3, 180, 30);
    uint64_t w3_8 = getbits(sf3, 210, 30);
    uint64_t w3_9 = getbits(sf3, 240, 30);
    uint64_t w3_10= getbits(sf3, 270, 30);

    // Omega0 (32 бита)
    uint32_t Omega0_raw =
        (((w3_4 >> 6) & 0xFFFFFF) << 8) |
        (((w3_5 >> 22) & 0xFF));
    ep->Omega0 = sbits(Omega0_raw, 32) * pow(2, -31) * 180.0;
```

```
    // Cis (16 бит)
    ep->Cis = sbits((w3_5 >> 6) & 0xFFFF, 16) * pow(2, -29);

    // i0 (32 бита)
    uint32_t i0_raw =
        (((w3_6 >> 6) & 0xFFFFFF) << 8) |
        (((w3_7 >> 22) & 0xFF));
    ep->i0 = sbits(i0_raw, 32) * pow(2, -31) * 180.0;

    // Crc (16 бит)
    ep->Crc = sbits((w3_7 >> 6) & 0xFFFF, 16) * pow(2, -5);

    // omega (32 бита)
    uint32_t omega_raw =
        (((w3_8 >> 6) & 0xFFFFFF) << 8) |
        (((w3_9 >> 22) & 0xFF));
    ep->omega = sbits(omega_raw, 32) * pow(2, -31) * 180.0;

    // OmegaDot (24 бита)
    uint32_t OmegaDot_raw =
        (((w3_9 >> 6) & 0xFFFF) << 8) |
        (((w3_10 >> 22) & 0xFF));
    ep->OmegaDot = sbits(OmegaDot_raw, 24) * pow(2, -43) * 180.0;

    // iDot (14 бит)
    ep->iDot = sbits((w3_10 >> 8) & 0x3FFF, 14) * pow(2, -43) * 180.0;

    return 0;
}

// ---------- Вывод в файл (формат Букалова) ----------
static void printEmp(const Ephemeris* ep) {
    FILE* out = fopen("out.txt", "w");
    if (!out) return;
    fprintf(out, "LNAV Ephemeris (slot = %u) =\n", ep->slot);
    fprintf(out, "Crs    = %.6e\n", ep->Crs);
    fprintf(out, "Dn    = %.6e    [deg/s]\n", ep->Dn);
    fprintf(out, "M0    = %.6f    [deg]\n", ep->M0);
    fprintf(out, "Cuc    = %.6e\n", ep->Cuc);
    fprintf(out, "e    = %.6e\n", ep->e);
    fprintf(out, "Cus    = %.6e\n", ep->Cus);
    fprintf(out, "sqrta   = %.6e\n", ep->sqrtA);
    fprintf(out, "toe    = %.0f\n", ep->toe);
    fprintf(out, "Cic    = %.6e\n", ep->Cic);
    fprintf(out, "Omega0  = %.6f    [deg]\n", ep->Omega0);
    fprintf(out, "Cis    = %.6e\n", ep->Cis);
    fprintf(out, "i0    = %.6f    [deg]\n", ep->i0);
    fprintf(out, "Crc    = %.6e\n", ep->Crc);
    fprintf(out, "omega  = %.6f    [deg]\n", ep->omega);
    fprintf(out, "OmegaDot = %.6e    [deg/s]\n", ep->OmegaDot);
    fprintf(out, "iDot   = %.6e    [deg/s]\n", ep->iDot);
    fprintf(out, "Tgd    = %.6e\n", ep->Tgd);
    fprintf(out, "toc    = %.0f\n", ep->toc);
    fprintf(out, "af2    = %.6e\n", ep->af2);
    fprintf(out, "af1    = %.6e\n", ep->af1);
    fprintf(out, "af0    = %.6e\n", ep->af0);
    fprintf(out, "WN    = %u\n", ep->WN);
    fprintf(out, "IODC   = %hu\n", ep->iodc);
    fprintf(out, "URA   = %hhu\n", ep->ura);
    fprintf(out, "Health = %hhu\n", ep->health);
    fprintf(out, "IODE2  = %hhu\n", ep->IODE2);
```

```
    fprintf(out, "IODE3  = %hhu\n", ep->IODE3);
    fprintf(out, "codeL2 = %hhu\n", ep->codeL2);
    fprintf(out, "L2P    = %hhu\n", ep->L2P);
    fclose(out);
}

int main() {
    char sf1[SUBFRAME_LEN+1] = {0};
    char sf2[SUBFRAME_LEN+1] = {0};
    char sf3[SUBFRAME_LEN+1] = {0};
    uint32_t slot = 0;

    if (file2subFrames("in.txt", sf1, sf2, sf3, &slot) != 0) {
        printf("Subframes for PRN 24 not found in in.txt!\n");
        return 1;
    }

    Ephemeris ep;
    memset(&ep, 0, sizeof(ep));
    ep.slot = slot;

    if (subFrames2Eph(&ep, sf1, sf2, sf3) != 0) {
        printf("Failed to decode ephemeris.\n");
        return 1;
    }

    printEmp(&ep);
    printf("Ephemeris successfully written to out.txt\n");
    return 0;
}
```

Этап 2:

Этап 2. Моделирование траектории движения

Эфемериды - параметры некоторой модели движения спутника. В разных ГНСС эти модели разные, а значит отличается и формат эфемерид, и алгоритмы расчета положения спутника.

Одна из самых простых и удобных моделей - в системе GPS.

Требуется:

1. реализовать на языке Matlab или Python функцию расчета положения спутника GPS на заданный момент по шкале времени UTC. В качестве эфемерид использовать данные, полученные на предыдущем этапе.
2. построить трехмерные графики множества положений спутника GPS с системным номером, соответствующим номеру студента по списку. Графики в двух вариантах: в СК ECEF WGS84 и соответствующей ей инерциальной СК. Положения должны соответствовать суточному интервалу на дне формирования наблюдений, определенном на предыдущем этапе. Допускается использовать одни и те же эфемериды на весь рассматриваемый интервал.
3. вывести значения координат спутника в файл out.txt в системе ECEF WGS 84 в виде строк: Секунда_от_начала_дня X Y Z.
4. используя оценку местоположения с предыдущего этапа, построить Sky Plot за указанный временной интервал и сравнить результат с Trimble GNSS Planning Online.

Оформить отчет по результатам этапа:

1. Реализация в Matlab или Python (описание модели и её листинг)
2. Таблица использованных эфемерид - *использовались из первого этапа*

3. Трехмерные графики положений спутника в ECEF и ECI (не забудьте подписать оси, изобразите соответствующую Земле сферу в начале СК)
4. Расчётный и полученный в GNSS Planing Online SkyView.

Таблица использованных эфемерид:

Crs = 2.275000e+01

Dn = 5.210052e-07 [deg/s]

M0 = -12.755586 [deg]

Cuc = -1.557082e-06

e = 1.636439e-02

Cus = 6.183982e-06

sqrtA = 5.153701e+03

toe = 288000 [s]

Cic = -2.142042e-07

Omega0 = -147.307115 [deg]

Cis = 1.266599e-07

i0 = 53.402052 [deg]

Crc = 1.897500e+02

omega = 82.744271 [deg]

OmegaDot = -4.756403e-07 [deg/s]

iDot = -4.752011e-08 [deg/s]

Tgd = -1.071021e-08

toc = 288000

af2 = 0.000000e+00

af1 = 2.159915e-12

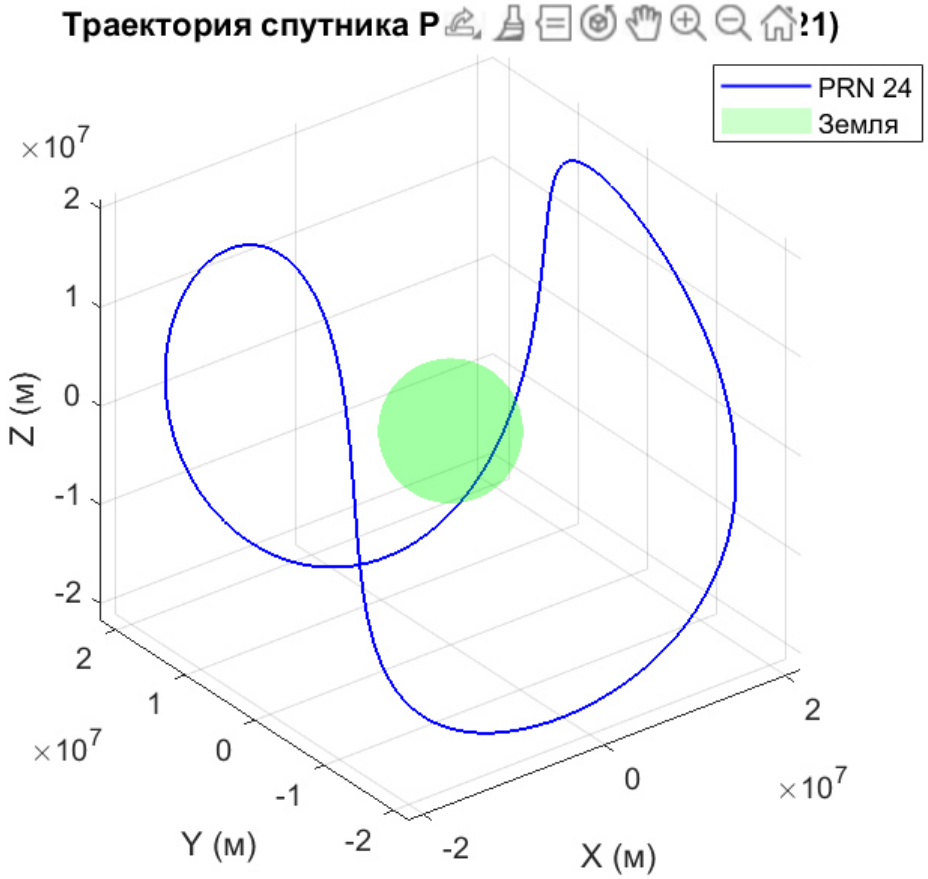
af0 = -2.265218e-04

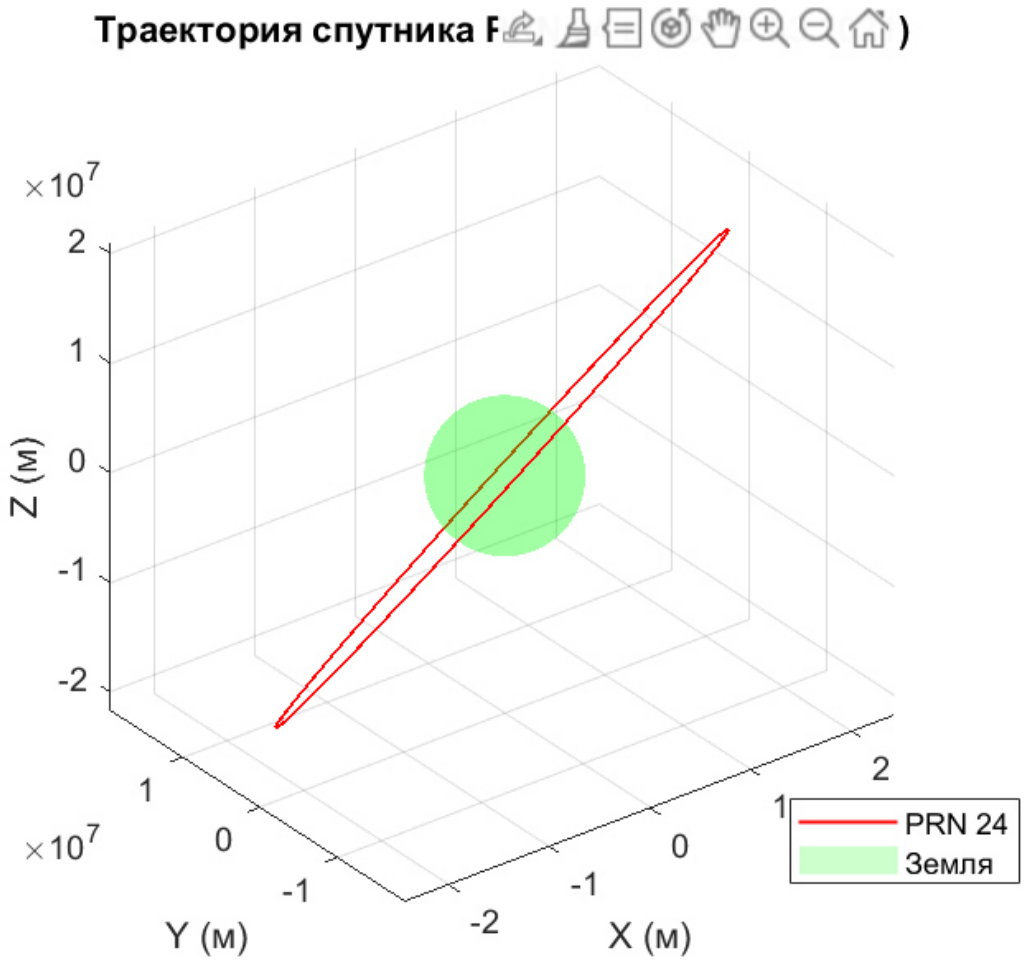
WN = 37

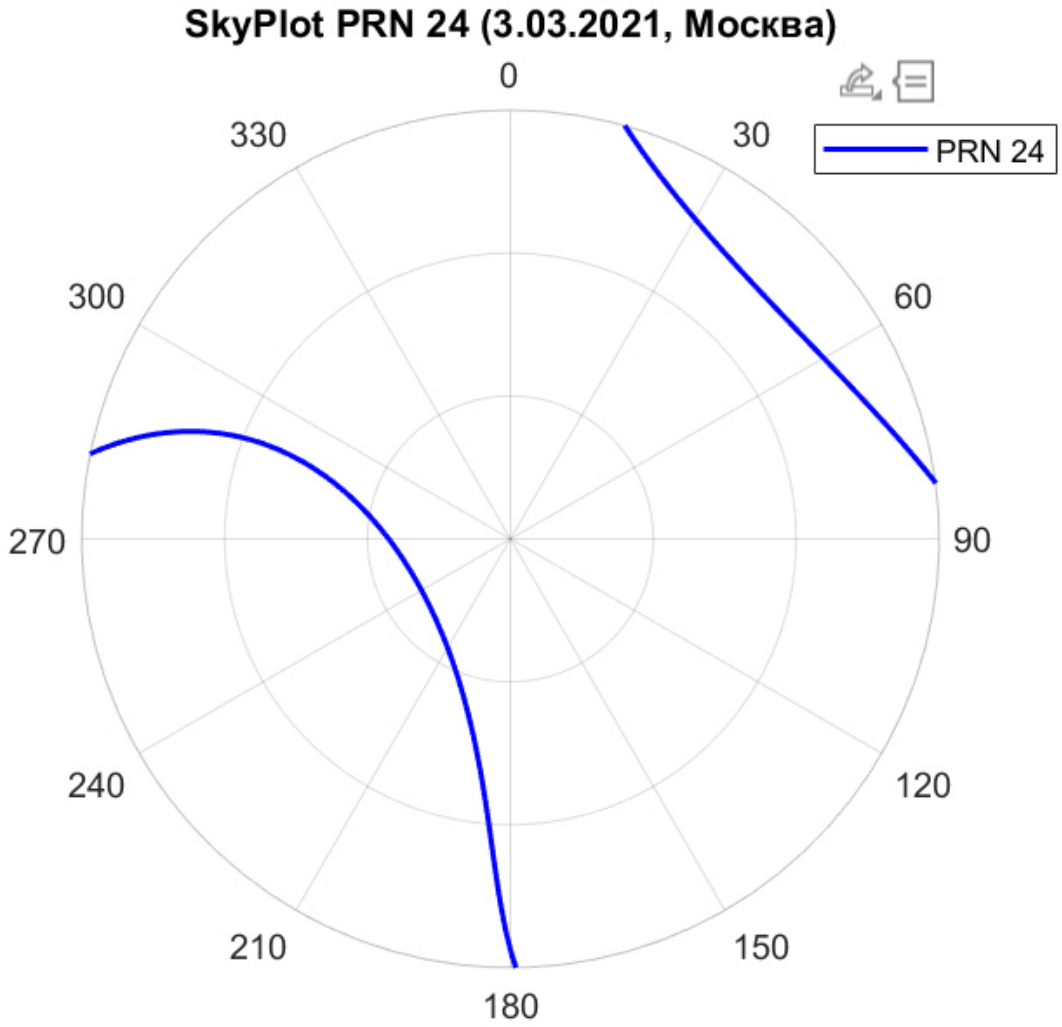
IODC = 7

URA = 0

Health = 0




Траектория спутника P)
×10^7
2
1
0
-1
-2
Z (м)
1
0
-1
×10^7
Y (м)
-2
-1
0
1
2
X (м)
PRN 24
Земля


SkyPlot PRN 24 (3.03.2021, Москва)
0
30
60
90
120
150
180
210
240
270
300
330
PRN 24

```
% Этап 2: Моделирование траектории спутника GPS PRN 24
% Эфемериды взяты из out.txt (Этап 1)
clear; clc; close all;

% ----- Эфемериды спутника 24 -----
ephem.Crs      = 2.275000e+01;
ephem.Dn       = 5.210052e-07;     % град/с
ephem.M0       = -12.755586;       % град
ephem.Cuc      = -1.557082e-06;
ephem.e        = 1.636439e-02;
ephem.Cus      = 6.183982e-06;
ephem.sqrtA    = 5.153701e+03;
ephem.toe      = 288000;           % секунды недели GPS
ephem.Cic      = -2.142042e-07;
ephem.Omega0   = -147.307115;      % град
ephem.Cis      = 1.266599e-07;
ephem.i0       = 53.402052;        % град
ephem.Crc      = 1.897500e+02;
ephem.omega    = 82.744271;        % град
ephem.OmegaDot = -4.756403e-07;    % град/с
ephem.iDot     = -4.752011e-08;    % град/с
ephem.toc      = 288000;
ephem.af2      = 0.0;
ephem.af1      = 2.159915e-12;
ephem.af0      = -2.265218e-04;

% ----- Константы -----
GM   = 3.986005e14;              % гравитационная постоянная Земли
OMGE = 7.2921151467e-5;          % угловая скорость Земли, рад/с
DEG2RAD = pi/180;
R_EARTH = 6371000;               % средний радиус Земли, м

% ----- Параметры наблюдателя (Москва, МЭИ) -----
lat = 55.7540 * DEG2RAD;
lon = 37.6200 * DEG2RAD;
h_obs = 200;                     % высота над эллипсоидом, м

% ----- Интервал моделирования -----
% 3 марта 2021 г. (день недели 3), начало дня в секундах недели = 3*86400
start_sec = 259200;
end_sec   = 345600;
dt = 300;                        % шаг 5 минут
t_week = start_sec:dt:end_sec;   % секунды недели

% ----- Расчёт всех положений -----
N = length(t_week);
x_ecef = zeros(1, N);
y_ecef = zeros(1, N);
z_ecef = zeros(1, N);

for i = 1:N
    [x_ecef(i), y_ecef(i), z_ecef(i)] = satpos(ephem, t_week(i));
end

% ----- Запись координат в файл out_xyz.txt -----
sec_of_day = (t_week - start_sec);   % секунды от начала суток
fid = fopen('out_xyz.txt', 'w');
for i = 1:N
    fprintf(fid, '%d %.3f %.3f %.3f\n', sec_of_day(i), x_ecef(i),
y_ecef(i), z_ecef(i));
end
```

```
fclose(fid);
disp('out_xyz.txt записан.');

% ----- 3D-график в ECEF -----
figure('Color','white');
plot3(x_ecef, y_ecef, z_ecef, 'b-', 'LineWidth', 1);
hold on;
% Сфера Земли
[u, v] = meshgrid(linspace(0, 2*pi, 60), linspace(0, pi, 60));
xs = R_EARTH * cos(u) .* sin(v);
ys = R_EARTH * sin(u) .* sin(v);
zs = R_EARTH * ones(size(u)) .* cos(v);
surf(xs, ys, zs, 'FaceAlpha', 0.2, 'EdgeColor', 'none', 'FaceColor',
'g');
xlabel('X (м)');
ylabel('Y (м)');
zlabel('Z (м)');
title('Траектория спутника PRN 24 в ECEF (3.03.2021)');
legend({'PRN 24', 'Земля'}, 'Location', 'best');
axis equal;
grid on;
saveas(gcf, 'ecef_trajectory.png');
disp('График ECEF сохранён (ecef_trajectory.png).');

% ----- ECI (инерциальная СК) -----
theta = OMGE * (t_week - start_sec);   % угол поворота Земли от начала
дня
cos_th = cos(theta);
sin_th = sin(theta);
x_eci = x_ecef .* cos_th - y_ecef .* sin_th;
y_eci = x_ecef .* sin_th + y_ecef .* cos_th;
z_eci = z_ecef;

figure('Color','white');
plot3(x_eci, y_eci, z_eci, 'r-', 'LineWidth', 1);
hold on;
surf(xs, ys, zs, 'FaceAlpha', 0.2, 'EdgeColor', 'none', 'FaceColor',
'g');
xlabel('X (м)');
ylabel('Y (м)');
zlabel('Z (м)');
title('Траектория спутника PRN 24 в ECI (3.03.2021)');
legend({'PRN 24', 'Земля'}, 'Location', 'best');
axis equal;
grid on;
saveas(gcf, 'eci_trajectory.png');
disp('График ECI сохранён (eci_trajectory.png).');

% ----- SkyPlot -----
az = zeros(1, N);
el = zeros(1, N);
rx0 = R_EARTH * cos(lat) * cos(lon);
ry0 = R_EARTH * cos(lat) * sin(lon);
rz0 = R_EARTH * sin(lat);

for i = 1:N
    dx = x_ecef(i) - rx0;
    dy = y_ecef(i) - ry0;
    dz = z_ecef(i) - rz0;
    e = -sin(lon)*dx + cos(lon)*dy;
    n = -sin(lat)*cos(lon)*dx - sin(lat)*sin(lon)*dy + cos(lat)*dz;
```

```
    u = cos(lat)*cos(lon)*dx + cos(lat)*sin(lon)*dy + sin(lat)*dz;
    az(i) = atan2d(e, n);    % азимут в градусах
    az(i) = mod(az(i), 360);
    el(i) = asind(u / sqrt(e^2 + n^2 + u^2));  % угол места
end

figure('Color','white');
polarplot(deg2rad(az), 90 - el, 'b-', 'LineWidth', 1.5);
ax = gca;
ax.ThetaZeroLocation = 'top';
ax.ThetaDir = 'clockwise';
ax.RLim = [0 90];
ax.RTick = [30 60 90];
% Убираем числовые метки у радиальных делений (если нужно)
set(ax, 'RTickLabel', []);
title('SkyPlot PRN 24 (3.03.2021, Москва)');
legend('PRN 24', 'Location', 'best');
grid on;
saveas(gcf, 'skyplot_matlab.png');
disp('SkyPlot сохранён (skyplot_matlab.png).');

disp('Все графики построены.');

% ----- Функция расчёта положения спутника -----
function [xECEF, yECEF, zECEF] = satpos(ephem, t)
    GM = 3.986005e14;
    OMGE = 7.2921151467e-5;
    DEG2RAD = pi/180;

    tk = t - ephem.toe;
    if tk > 302400
        tk = tk - 604800;
    elseif tk < -302400
        tk = tk + 604800;
    end
    n0 = sqrt(GM / ephem.sqrtA^6);
    n = n0 + ephem.Dn * DEG2RAD;
    M = ephem.M0 * DEG2RAD + n * tk;
    % Уравнение Кеплера
    E = M;
    for j = 1:10
        E_old = E;
        E = M + ephem.e * sin(E_old);
        if abs(E - E_old) < 1e-12
            break;
        end
    end
    v = 2 * atan2(sqrt(1+ephem.e)*sin(E/2), sqrt(1-ephem.e)*cos(E/2));
    phi = v + ephem.omega * DEG2RAD;
    du = ephem.Cus*sin(2*phi) + ephem.Cuc*cos(2*phi);
    dr = ephem.Crs*sin(2*phi) + ephem.Crc*cos(2*phi);
    di = ephem.Cis*sin(2*phi) + ephem.Cic*cos(2*phi);
    u = phi + du;
    r = ephem.sqrtA^2 * (1 - ephem.e*cos(E)) + dr;
    i_orbit = ephem.i0*DEG2RAD + di + ephem.iDot*DEG2RAD*tk;
    x_orb = r * cos(u);
    y_orb = r * sin(u);
    OMEGA = ephem.Omega0*DEG2RAD + (ephem.OmegaDot*DEG2RAD - OMGE)*tk -
OMGE*ephem.toe;
    xECEF = x_orb*cos(OMEGA) - y_orb*cos(i_orbit)*sin(OMEGA);
    yECEF = x_orb*sin(OMEGA) + y_orb*cos(i_orbit)*cos(OMEGA);
```

```
    zECEF = y_orb*sin(i_orbit);
end
```